

Assignment 5: Quicksort Algorithm: Implementation, Analysis, and Randomization

Alexandria Roberts

School of Computer and Information Sciences

MSCS531-A01 Computer Architecture and Design

Professor Bass

11/15/2025

Introduction

For this assignment, I implemented two versions of the Quicksort algorithm in Python. One was a deterministic version and a randomized version. Along with their time and space complexity and compared their performance on different types of input arrays analyzed.

Deterministic Quicksort

The deterministic version at all times chooses the last element as the pivot. The functions implemented includes `partition(arr, low, high)` and `quicksort(arr, low, high)`

- `Partition(arr, low, high)`- rearranges elements where values $\leq$ pivot move left, and values $>$ pivot move right.
- `Quicksort(arr, low, high)`- recursively sorts both the left and right partitions.

This version seems to work correctly but has performance issues on sorted or reverse-sorted inputs due to unbalanced pivot choices.

Randomized Quicksort

The randomized version chooses a random pivot index, then swaps that element into the last position, and then partitions the array. Functions implemented includes `random_partition(arr, low, high)` and `randomized_quicksort(arr, low, high)`.

- `Random_partition(arr, low, high)`- will select a random pivot.
- `Randomized_quicksort(arr, low, high)`- Recursively sorts using random pivots. This helps reduce the chance of hitting worst-case behavior.

#Time and Space Complexity

Case	Deterministic	Randomized
Best	$O(n \log n)$	$O(n \log n)$
Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n^2)$, but very unlikely

Space Complexity

Average: $O(\log n)$ recursion depth

Worst: $O(n)$ when recursion becomes unbalanced

Empirical Comparison

I was able to test both Quicksort versions on arrays of sizes 100 and 500 using three input types: random, sorted, reverse-sorted.

Empirical Comparison: Deterministic vs Randomized Quicksort Results

Size	Case	Deterministic (s)	Randomized (s)
------	------	-------------------	----------------

100	random	0.000044	0.000068
100	sorted	0.000260	0.000085
100	Reversed-sorted	0.000192	0.000085
500	random	0.000301	0.000430
500	sorted	0.006711	0.000447
500	reversed-sorted	0.005438	0.000433

Interpretation of the results shows random input; both versions perform similarly. Sorted and reverse-sorted input, deterministic quicksort slows down significantly. Randomized Quicksort stays consistently fast; this is most likely because random pivots avoid bad patterns.

Conclusion

This assignment shows that pivot choice does indeed affect Quicksort's performance. The deterministic version while predictable but vulnerable to worst-case behavior. On the other hand the randomized version provides more stable and reliable performance across all input types. To avoid poor pivot choices in practice randomized quicksort is the better option.